PROJECT AND TEAM INFORMATION

Project Title

Smart Healthcare Appointment System

Student / Team Information

Team Name-DSCPP-III-2026-T275	*Team Members*
Team member 1 (Team Lead) *(Holariya, Raunak: 2510012150: 2510012150@geu.ac.in)*	*Holariya, Raunak – 9548477871* *2510012150@geu.ac.in* 
Team member 2 *(Ronit pal: 2510019652: 2510019652@geu.ac.in):*	*Ronit pal – 9258439739* *2510019652@geu.ac.in* 
Team member 3 *(Kartik Gaur: 2510360058: 25100360058@geu.ac.in)*	*Kartik Gaur – 925953745* *25100360058@geu.ac.in* 

PROPOSAL DESCRIPTION

Motivation

i-Improving Patient Care: Long waiting times and mismanaged appointments are common in hospitals. A smart system ensures faster scheduling and prioritizes emergency cases.
ii-Efficient Resource Use: Doctors' time and hospital resources can be better managed with automated scheduling and workload distribution.
iii-Accessibility: Patients can register and book appointments easily, reducing paperwork and confusion.
iv-Data-Driven Insights: Collecting patient and doctor data allows analysis of hospital performance, average waiting times, and patient flow.
v-Learning Perspective: For students, it's a chance to apply queues, linked lists, recursion, and OOP in a real-world inspired scenario.

State of the Art / Current solution

The problem of inefficient healthcare management is solved by designing a system that integrates patient registration, doctor scheduling, and appointment handling into a single framework. Patients are registered into queues, with emergency cases prioritized using a priority queue to ensure immediate attention. Doctors' schedules are maintained as linked lists of available time slots, allowing dynamic assignment of patients to suitable doctors. Appointments can be booked, cancelled, or rescheduled, while recursive functions calculate average waiting times and reassign slots when conflicts occur. The system also generates reports on patient flow, doctor workload, and hospital performance. By combining queues, linked lists, recursion, and OOP principles, the Smart Healthcare System reduces waiting times, optimizes resource utilization, and ensures that emergency cases are handled promptly, thereby making healthcare delivery smarter, faster, and more reliable.

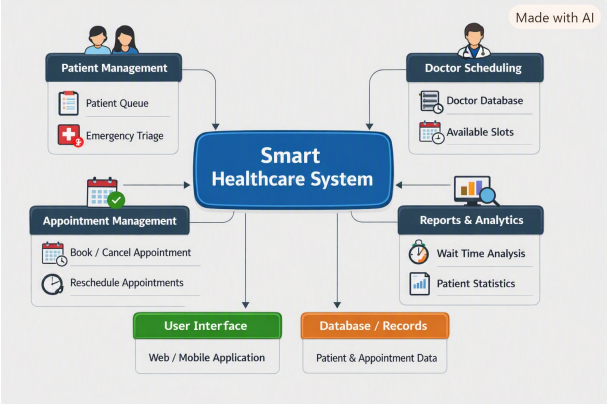
Project Goals and Milestones

i-Requirement Analysis and Design – Identify key problems in hospital workflows and design the system architecture, including patient queues, doctor schedules, and appointment handling.
ii-Class and Data Structure Implementation – Develop classes for patients, doctors, and appointments, and implement queues, linked lists, and priority queues to manage data efficiently.
iii-Core Functionality Development – Build modules for patient registration, doctor scheduling, appointment booking, cancellation, and rescheduling.
iv-Statistics and Reporting – Implement recursive functions to calculate average waiting times and generate reports on patient flow and doctor workload.
v-Testing and Validation – Test the system with sample data to ensure correctness, efficiency, and reliability in handling different scenarios, including emergencies.
vi-Final Documentation and Presentation – Prepare a detailed project report with motivation, problem-solution, flowcharts, algorithms, and results, followed by a presentation to demonstrate the system's impact.

Project Approach

The Smart Healthcare System is approached through a structured methodology that combines problem analysis, system design, and implementation using data structures and object-oriented programming. The first step involves identifying the key challenges in hospital workflows, such as patient waiting times, emergency prioritization, and doctor scheduling. Based on this analysis, the system architecture is designed to incorporate queues for patient management, linked lists for doctor schedules, and priority queues for emergency cases. Object-oriented principles are applied to create modular classes for patients, doctors, and appointments, ensuring scalability and clarity in design.
The implementation phase focuses on developing core functionalities such as patient registration, appointment booking, cancellation, and rescheduling. Recursive functions are integrated to calculate average waiting times and generate statistical reports, thereby enhancing the analytical capability of the system. Testing is carried out with sample data to validate correctness, efficiency, and reliability under different scenarios, including emergency handling. Finally, the project is documented with flowcharts, pseudocode, and performance analysis, and presented as a complete solution that demonstrates both technical mastery and social relevance.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The Smart Healthcare System successfully demonstrates how technology can streamline hospital operations and improve patient care. The system reduces waiting times by prioritizing emergency cases through a priority queue, ensures efficient doctor scheduling using linked lists, and simplifies appointment management through object-oriented design. It provides accurate reports on patient flow and doctor workload, enabling data-driven decision-making. Academically, the project achieves its goal of applying data structures such as queues, linked lists, and recursion in a real-world context, while practically showcasing how automation can enhance healthcare efficiency and reliability.

Assumptions

The Smart Healthcare System is developed under several logical and technical assumptions to ensure its effective functioning. It is assumed that all patient and doctor data entered into the system are accurate and valid, with no duplicate or missing records. The hospital has a stable digital infrastructure capable of supporting the system's operations, including data storage and retrieval. Each doctor's schedule is predefined and updated regularly, and patients provide complete information during registration. Emergency cases are correctly flagged to enable priority handling. It is also assumed that users—both patients and hospital staff—possess basic computer literacy to interact with the system's interface. Furthermore, the system operates in a controlled environment where network connectivity and hardware resources are sufficient for smooth execution. These assumptions help maintain consistency, reliability, and accuracy throughout the system's workflow.

Reference

GeeksforGeeks. Data Structures and Algorithms in C++ — concepts of queues, linked lists, and priority queues used for patient and doctor management.
TutorialsPoint. Object-Oriented Programming in C++ — class design and modular structure for system components.
ResearchGate. Smart Healthcare Systems: A Review of Technologies and Applications — insights on healthcare automation and digital scheduling.
IEEE Xplore. Hospital Management System Using Data Structures — reference for workflow and appointment handling logic.
Microsoft Learn. C++ Programming Fundamentals — guidance for implementing efficient and scalable code architecture.